

University of Plymouth

**SCHOOL OF ENGINEERING,
COMPUTING, AND MATHEMATICS**

**COMP3000
COMPUTING PROJECT
2025/2026**

DEVELOPING A TURN-BASED COMBAT GAME AS A TOOL FOR STRATEGIC SKILL ACQUISITION

Jonathan Michael Andrews

10849920

BSc (Hons) Computer Science (Games Development)

1.1 Acknowledgements

The author would like to thank their supervisor for their guidance and feedback throughout the duration of this project. Appreciation is also extended to all participants of user experience testing who provided valuable insights that contributed to the evaluation and improvement of the system.

1.2 Abstract

This project presents the development of a game-based learning system designed to support the development of strategic thinking skills in users aged 10-14. Traditional educational approaches often struggle to effectively teach skills such as planning, evaluation, and adaptive decision making, particularly in engaging formats. To address this, a turn-based combat game was designed and implemented using the Unity engine, allowing players to formulate, test and refine strategies in a low-risk environment.

The system incorporates modular architecture, including combat management, AI decision making, and an ability-based progression system, enabling scalable and iterative development. An agile methodology was adopted, supporting incremental development and continuous refinement.

Evaluation was conducted through automated testing, playtesting, and user experience testing with participants unfamiliar with the system. Results indicated that the game was generally accessible and engaging, with users reporting a high level of clarity and ease of interaction. However, limitations were identified in the sample size and short evaluation period, restricting the ability to measure long-term skill development.

Overall, this project demonstrates that a structured, turn-based game can provide an effective and engaging environment for practicing strategic thinking skills, with opportunities for further refinement and expanded evaluation in future work.

Contents

Acknowledgements.....	2
Abstract.....	3
Word count	5
Code link	5
1Introduction	6
1.1Problem domain.....	6
1.2Project overview	7
1.3.Aim	8
1.4Objectives:	9
1.5Scope.....	10
2Background & Literature Review	11
2.1Background	11
2.2Games as experimental learning tools.....	12
2.3Strategy games and cognitive skill development	13
2.4Learning and engagement	14
2.5Existing work	15
2.6Summary of findings	16
3Project Definition & Planning	17
3.1Project scope	17
3.2Requirements.....	18
3.3Target users	19
3.4Objectives.....	20
3.5Project management	21
3.6Tools and technologies	22
3.7Risk assessment	23
4Methodology	30
4.1Alternatives considered	30
4.2Development approach.....	31
4.3Evolution of requirements	32
4.4Feedback incorporation.....	33
5Implementation	34
5.1Architecture	34
5.2Core systems.....	36
6Testing & Evaluation	44
6.1Testing strategies.....	44
6.2User experience testing.....	48
6.3Discussion.....	49
7Conclusion & Future Work.....	50

7.1Conclusions	50
7.2Future work	51
8References:	52

Figure 1-Initial Gantt chart	26
Figure 2 - Final Gantt chart	27
Figure 3 - Sprint backlog (Trello board)	29
Figure 4 - High-level system architecture design	34
Figure 5 - Unity scene hierarchy and flow	35
Figure 6 - Combat scenario UI	36
Figure 7 - Combat manager implementation	37
Figure 8 - Ability SO example	38
Figure 9 - Ability behaviour code example (Buff effect)	39
Figure 10 - Damage calculation scaling graph	40
Figure 11 - Dungeon generation example	42
Figure 12 - UTF test results	44
Figure 13 - UTF test example	45
Figure 14 - Bug log extract	46

1.2.1 Word count

10,999

1.2.2 Code link

<https://github.com/JAndrews004/COMP3000-JonathanAndrews>

2 Introduction

2.1 Problem domain

Throughout this project a game-based learning system was developed using the Unity engine. The product is a turn-based combat game with skills and stats to allow for player customisation to the experience. The game is rule-driven with progression in difficulty throughout. A game-based application was used as the skills being targeted are difficult to develop through traditional education as shown in Gouveia, Lopes and Vaz, (2011). This problem is relevant to improving skills related to formulating, testing, and evaluating strategies in a rule-based environment. Strategic thinking can help with planning, adaptive thinking, and decision-making under constraints. These skills are used regularly in careers especially in management roles and can transfer to different settings for example in sports it can help you find strengths and weaknesses in the opposition and yourself. Making these skills valuable for a range of real-world contexts.

2.2 Project overview

This project is a game where players must defeat enemies of varying degrees of difficulty ranging from random actions to coordinated attacks. The player will have a basic gameplay loop of entering a 'dungeon' defeating all the enemies within while making short- and long-term strategies. These can also be refined in the hub world where new abilities can be bought and allows the player to make a team fitting their strategy. Each character the player controls will have stats that also contribute to different aspects of gameplay such as damage or defence, making even more possibilities for numerous strategies possible. All these actions will be diligently designed and evaluated not just implemented. Throughout the game players must formulate strategies to use during combat by choosing what abilities to equip. They will need to evaluate how these abilities synergise to be able to improve their strategies over time. They will then test these strategies to ensure they work effectively in the combat scenarios and determine how they could be improved. This project is a technical implementation involving systems, architecture, testing, and evaluation.

2.3 Aim

The aim of this project is to teach people 10-14 years old strategic skills in a format which they will be engaged in and motivated to continue. The goal is to teach them without using traditional teaching techniques. As an evaluation of skill improvement would require a longer timeframe than this project allows success will be determined through the players enjoyment and the accessibility to inexperienced players. Both will be assessed through user experience testing and accessibility will be defined as how easy it is for a new player to understand and start playing the game. Supporting evidence will be discussed in the literature review to show evidence that this system may work in improving these skills.

2.4 Objectives:

- Make a functional and robust game that passes stringent testing.
- Have the game's core function be based on strategic thinking and actions.
- Support players learning of core strategy development skills.
- Make the theme of the game engaging and appropriate for 10–14-year-olds by following industry standards.

2.5 Scope

This report will go through the background of this project and the literature to support its existence. It will also include information on the project's definition, planning, methodology and implementation. Then it will move onto the testing, evaluation, and discussion. Finally, the conclusion of this project and any future work for this project.

3 Background & Literature Review

3.1 Background

In video games strategic thinking is used for both short- and long-term gameplay. Some games, such as MegaCrit's Slay the Spire and Subset Games' Into the Breach, focus on quick tactics and plans that allow players to iterate over a brief period of time to improve their performance. Other games, such as Paradox Interactive's Stellaris and Paradox Interactive's Crusader Kings III, focus more on long-term tactics with an emphasis on in-depth planning and thinking ahead. Both formats can improve the formulation, testing and evaluation of strategies over a long-time frame in an enjoyable environment.

Turn-based systems are used to support short- and long-term gameplay further as it enables deeper planning and evaluation, allowing players to learn through reflection. These decision-making moments provide players with time to adjust their strategies and tactics to best respond to in-game scenarios.

Team management is used to increase strategic depth in single player games, such as Larian Studios' Divinity: Original Sin 2, as the player not only needs to think about each individual character but also how they synergise to produce the overall action for the team. This can lead to players identifying strengths and weaknesses in each character and finding ways to minimise risk and maximise their potential gains.

3.2 Games as experimental learning tools

Games are effective as experimental learning tools; this is due to enabling risk-free trial and error.

In Gouveia, Lopes and Vaz, (2011) the learning of concepts through trial and error was shown. This is relevant to games as they allow users to experience a variety of strategic situations where different knowledge is needed. Knowledge of physics concepts used in engineering were measured using questions after the use of a simulator and compared to previous students' results. This allowed the students to apply what they learned in different scenarios and build their understanding of the topic and apply it to questions based on their experiences.

Furthermore, Saenz and Cano, (n.d.) showed iterative learning through making mistakes. Subjects experienced a project management simulation, and learning was measured from feedback using questionnaires after undertaking a real project to see the extent to which the knowledge acquired was applied. Subjects who performed worse in the simulator showed greater experimental learning due to being able to learn what does not work and the reasons for it. This can then be applied to similar situations to know what plans will not work and the reasoning for it. This suggests that mistakes are a crucial part of the learning process. This is facilitated by low-risk environments as users feel less pressure to avoid mistakes.

Finally, Ho et al., (2022) measured perception, knowledge and attitude through pre-test and post-test questionnaires. A digital and non-digital board game based around the Sustainable Development Goals from the United Nations was used as a learning tool. Both groups saw improvements to all aspects with a slightly larger increase for digital. This was suggested in the study to be due to the physical version having a slower pace. This would then lead to a slower pace of skill application.

This suggests that environments that promote experimentation allow players to improve their skills more effectively as there is less risk to failure. However, the games produced in these papers were hyper specific to the skills being targeted. This means the designs of these games were tailored to maximise learning of these specific skills. This could mean that games aren't the reason for the learning but the design choices through their development. This does not mean that there is no learning involved in more generalised games. This transfer occurs as it involves shared cognitive processes such as hypothesis testing, planning, and evaluation. These skills are crucial to experimentation as they are needed to iterate on successes and improve failures. This suggests that for this project having a low-risk environment will promote experimental thinking and therefore improve learning in the skills being used. This project is tailored for this by allowing retries and having no penalties for failure which will allow for iteration and learning.

3.3 Strategy games and cognitive skill development

Strategy games are believed to allow for enhanced cognitive skill development by requiring players to analyse multiple factors and outcomes.

This is shown in Bottino et al., (2007) where children of primary school age used video games to enhance their logical and strategic reasoning skills. This learning occurred through specially designed games that were classified as brainteasers or puzzlers. This allowed the subjects to plan out a strategy and evaluate why it failed or succeeded. However, one downside to this was that the subjects struggled to comprehend the task and to construct an initial strategy. This was believed to be due to the different and sometimes interwoven abilities; for example, some subjects struggled to formulate testable hypotheses to guide their strategies. The results of this paper were measured using INVALSI (Italian National Evaluation Institute of the Ministry of Education) mathematics test and compared to two untested classes. The results suggest a meaningful increase in performance.

Strategy games enhancing cognitive skill development is also shown in Glass, Maddox, and Love, (2013) in which participants with little to no gaming experience were tested before and after forty hours of gameplay. Participants' cognitive skills were measured pre and post gameplay using cognitive testing. All participants however were female as there were insufficient male participants with little gaming experience. The results of this paper suggest that RTS (real time strategy) games promote cognitive flexibility, which is defined as "the mental ability to adapt thinking and behaviour in response to changing, unexpected, or new environmental demands." through many unexpected situations that require quick and decisive actions. RTS games promote this by having multiple complex agents that react to each other causing unique scenarios to occur.

Together these studies illustrate that strategy games improve both structured problem-solving in children and adaptive thinking in adults. For this project I will use a more structured combat system to leverage the improvements seen in the first study while using some randomness to include adaptive thinking seen in the second study. This should enhance the cognitive skill development this project aims to achieve.

3.4 Learning and engagement

For a game to successfully facilitate learning it must have high engagement. Engagement and motivation are intricately linked; therefore, the player must be motivated to play and learn.

This is shown in Pesare et al., (2016) where users' motivation and engagement were measured pre- and post-testing. Engagement can be in many forms, such as physically, cognitively, and emotionally when participating in a task. This involvement can present itself as focus and passion. In this study medical training, for both patient and physicians, was used in the form of a game. They decided to leverage all aspects of user investment to maximise motivation and knowledge gain. In all groups motivation and knowledge gain both increased. Motivation is tied to active participation and therefore learning. However, it was not as much as they expected for two main reasons. For the patients, their initial motivation was low so they may not have been giving as much attention to the game to increase their knowledge. For the physicians, their knowledge base was already high so there wasn't much room for improvement. However, both groups saw an increase in motivation which could have been leveraged in a longer study with expanded gameplay for the physicians to expand knowledge.

In Nadeem, Oroszlanyova and Farag, (2023) game technologies were used as an alternative format to traditional education. Game-based quizzes were used to measure this where a higher activity completion rate was seen compared to the non-game version. This suggests that the game-based version was more engaging to the students which is due to them being more interactive. However, in the latter half of the academic year the completion rate fell again. This may be due to there not being any new content to challenge the students. Therefore, for prolonged engagement, a challenge must exist for the player. This would cause the game to be too easy and therefore less engaging as they are no longer as cognitively engaged. However, the initial engagement improvements suggests that the game-based version is more engaging if it has long term support for the students to be challenged. This would lead to better learning performance as they are completing quizzes with more advanced knowledge.

These studies illustrate a need for engagement to have a successful learning outcome. Games are well suited for this as they promote active participation. They also provide a challenge to players and have a deep sense of progression. For my product I need to ensure that it is engaging and sets a challenge for the player. Engagement can also be leveraged through the visual style which will be considered next.

3.5 Existing work

Many existing design approaches could have been used for this product however they needed analysis to determine which was best suited as a solution.

For the visual style, a retro theme was chosen for its simplicity which allows for better readability of characters. It is also more accessible for younger audiences which would allow a wider range to play. This pixelated style also reduces the visual noise and therefore the cognitive load of playing the game. This will make the game more accessible for players with a lower skill level and aid in experimental learning as there are fewer distractions so players can focus more on developing and evaluating strategies. Because of this, gameplay systems must also support accessibility, player experience, and strategic depth. Different combat styles were explored to find one best suited to the aims of this project.

One combat system that was considered was a real-time combat experience. This is reaction-based combat with continuous input from the player which can reduce decision quality as shown in Maule and Edland, (2002). This is shown in games such as "Dark Souls" where the player's reaction time is extremely important as poor reactions are punished. Another combat system considered was turn-based with no movement. This combat system consists of players choosing actions with as much time as they want to think over decisions. Finally, a turn-based with movement was considered as it has the same features as the turn-based system but with a spatial decision-making element. The real-time combat system is more cognitively loaded and therefore less accessible as a baseline reaction time is needed to play correctly. The potential decision-making depth can be reduced as players fall into habits made through gameplay. However, this approach is more engaging for players as it is faster paced and can feel more exciting. A turn-based system is more reflection based and allows for players to consider actions in depth. Shaheen, Fotaris and Fallahkhair, (2021) shows that allowing time for reflection improves critical thinking, problem-solving, and retention rate. This allows players to form more complex strategies using deeper thinking and reflection. As there is no time pressure the player can make more focused decisions and focus on the strategy of the combat. However, the higher-level planning is increased significantly with the addition of spatial decision making. This would allow players to make more complex strategies. However, accessibility may be reduced as it would be harder to learn how to play and to get initial strategies formulated. Turn-based combat can feel slow and less exciting than real-time combat, but real-time combat can overwhelm players new to the genre. Because of this the turn-based with no movement was chosen to maximise strategic depth and approachability. The reduced cognitive load is also a benefit as it allows for more strategic thinking rather than reactive thinking which is the aim of the project. This will aid the learning outcomes and cognitive skill development this project is aiming to achieve.

3.6 Summary of findings

Overall, I decided that the best solution to this problem was a turn-based combat game. This is because games allow for greater learning of skills as discussed above. It would also allow for people to use with minimal training as games are very wide-spread and for a lot of people intuitive to play. I decided on a combat game as it makes the player must think about many aspects rather than just one. This would increase the potential difficulty and therefore the maximum learning that the game can provide. A combat game was also picked as its core progression is skilled based with some upgrade-based progression to aid in the learning process. This means that players would need to learn the game's systems and find their own solutions rather than investing a lot of time to upgrade and trivialise gameplay. A turn-based approach was decided on as it allows time for players to think actions through and plan thoroughly. This also allows players not accustomed to playing games enough time to think through what might be obvious to others. This approach also eliminates other variables such as reaction times that could limit some players progression.

4 Project Definition & Planning

4.1 Project scope

This project is primarily for learners aged 10-14, whose problem is learning key skills traditional education struggles to teach in an engaging format. "Plans & Peril" is a game that teaches long-term strategies and short-term tactics for a group with a variety of strengths and weaknesses. The game will focus on the skills required to formulate, test and evaluate strategies. It will encourage them how to utilise these skills to perform against challenging and varied situations. The turn-based structure was chosen to encourage reflective decision-making. This will help players to think more broadly about options by determining what is working and failing in their current attempt. The art-style is pixel art as this will have an aesthetic more suited for the target audience and any younger players. This will lead to more engagement while playing the game. The enemies will have distinct levels of 'intelligence' related to strategy to continually challenge the player with new ways of thinking to find flaws and use them to their advantage. This will support experimentation as new enemies will bring new challenges. The controls will be fully UI based as to allow the barrier of entry to be extremely low and opening the possibility for mobile support in the future. This will make the game more accessible for the target audience as they may not have had much experience with keyboard controls and may struggle to adapt.

4.2 Requirements

The overall aim of the project is to be an educational source for younger learners (age 10-14) to develop skills that are hard to develop through traditional education. This is due to a large amount of experimentation through practical application required. A game is the best format for this as it doesn't require more people to begin and has no adverse effects for failure so people can experiment as much as they wish. A low-risk environment allows for more experimentation without the fear of failure. The main skills I am aiming for this game to develop are centred around formulating and improving strategies through planning, reflection, and evaluation to reach actionable improvements.

4.3 Target users

This product being targeted at 10–14-year-olds, doesn't mean I wouldn't want older or younger people playing the game. To accommodate this, I am following the PEGI guidelines for a PEGI 3 game which is stated as 'not contain any inappropriate content but can sometimes be too difficult to master for younger children' this makes it easier to develop as I don't need to worry about the difficulty just the visual appropriateness. This pushed me more towards using a pixel-art style as it is easier to comply with this. As the primary users of this product will be children I have decided to not collect any personal data and as a result be following GDPR. The only data being saved will be to do with the players progress through the game. This is particularly relevant as my target audience is children and they generally have less awareness of their personal data. It is also a much higher danger if this data is misused. Having both in place will protect the players and follow the Online Safety Act. To ensure the safety of the players, as my ethical responsibility, I will follow these guidelines and laws. This game following more guidelines and being safer means that more parents are likely to let them play and therefore be more accessible.

4.4 Objectives

My primary objective for this project is to make a functional and robust game with minimal to no major bugs or crashes that prevent gameplay. This is because if this is not met then the product will not be functional and therefore useless. To ensure that this is met I will be testing the game throughout the project's duration and in different formats being a final build or in the editor. Another objective is to have the game's core function to be based on strategic thinking and actions. As this is the main aim of the game, the gameplay itself must allow for players to develop these skills in a practical way. Finally, I will make the theme of the game suitable for the targeted audience by following industry standards as discussed before. To ensure I meet this objective I am using a pixel art-style with no gore to guarantee it is suitable for my audience.

4.5 Project management

The development methodology I decided to use for this project was agile. I decided on this as it offers a very structured development lifecycle by working in sprints which makes it easier to know if the project is on schedule for completion or not. It also allows for quick iteration and implementation of feedback which will allow me to improve my project quickly. I used 2-week sprints where each sprint is planned at the end of the previous one so that the work to complete is accurate. If I was ahead of schedule at any point, I would research how to develop the next sprint to save some time later or I would do some polishing of game mechanics so that it wasn't left until the end.

To manage my time, I used a Gantt chart as shown in figure 1 where each sprints main objective is listed and each block of sprints' deliverables. This helped to plan my time efficiently and trim the scope of the project early on so as I knew roughly how much time I would have. This changed throughout the project due to some work taking a different amount of time than expected and the Gantt chart was updated as shown in figure 2. The main change was during block 12 the work required was a lot shorter than expected and the next sprint was also started and completed. So, for the next sprint a testing and polishing sprint was included to finish up any small pieces and fix any bugs encountered.

4.6 Tools and technologies

The tools I decided to use were GitHub for having a repository of my project so I didn't lose any data and to act as a development log where I can easily go back to earlier versions to get an understanding of the development at that time. I decided to use Trello for planning and decomposing the sprint into actionable development steps as shown in figure 3. This helped me have a good view on the amount of work required and let me plan easily. I decided to use Unity to build this project as I have experience with it but also with some other game engines such as Unreal Engine but Unity's systems are easier to understand so would make learning new technologies for this projects development quicker.

4.7 Risk assessment

Before planning the project in depth, a risk assessment was conducted to ensure that any major risks to development were addressed early on. Mitigations were decided upon to ensure that if any of the risks identified did occur a suitable plan was already in place. These risks were also reviewed every sprint to determine if the likelihood of it occurring had increased.

Risk score index 1 – 10, 1 safe, 10 extremely dangerous and likely

Risk	Category	Who might be harmed and how?	Mitigations in place	Risk score	Further action	Target risk score
Licensing agreements	Legal	License holder and me, by having their intellectual property used without consent and by having to remove assets already implemented	Using free assets with clear statement about usage for non-commercial and commercial products	3	Making my own assets for some items I can't find compatible for online	2
Inappropriate content	Legal	Target audience by causing harm though content that is too mature for them	Not including any gore or foul language	5	Following PEGI guidelines	3
Schedule over-run	Time	Users of the product by a delayed launch	Gantt Chart and rough sprint plan laid out, and 4 weeks set aside for 'polishing' can be used as extra development	4	Cut features if severely behind schedule.	2

Scope creep	Scope	Users of the product by having an incomplete product.	Scope locked down early and no changes unless original scope complete	6	Any additional ideas for features added to a backlog then near end of development decide based on impact and time required whether to implement.	4
System integration issues	Technical	Users of the product by having an incomplete or unfunctional system	Starting development on the most core features and integrating them from the start.	4	Testing system after each sprint to ensure functionality is still there. Also including automated testing to guarantee core logic is unaffected.	1
Mechanics not being engaging	User experience	Users of the product by not having an engaging and fun experience	Design mechanics using industry-standard mechanics that are already established.	6	During user testing evaluate the user's enjoyment and ask for where they would want to see improvements.	4
Balancing issues	User experience	Users of the product by having a frustrating experience.	Using equation-based scaling to ensure fairness.	4	Play-testing regularly to ensure balance and adjusting values and rules to improve balance.	3

Limited user testing sample	Testing	Producer of the product by not having enough insight before launch.	Attending user testing event to gather user testing data.	6	Ask further peers for feedback to increase sample size.	4
-----------------------------	---------	---	---	---	---	---

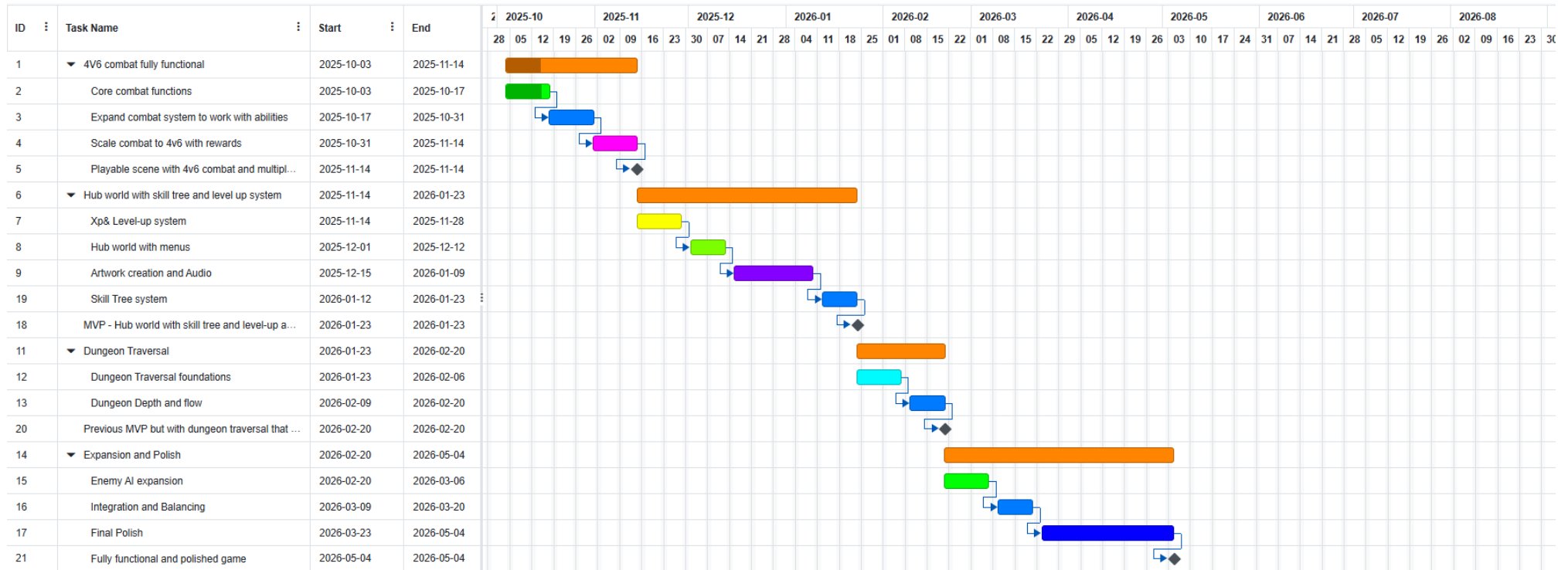


Figure 1-Initial Gantt chart

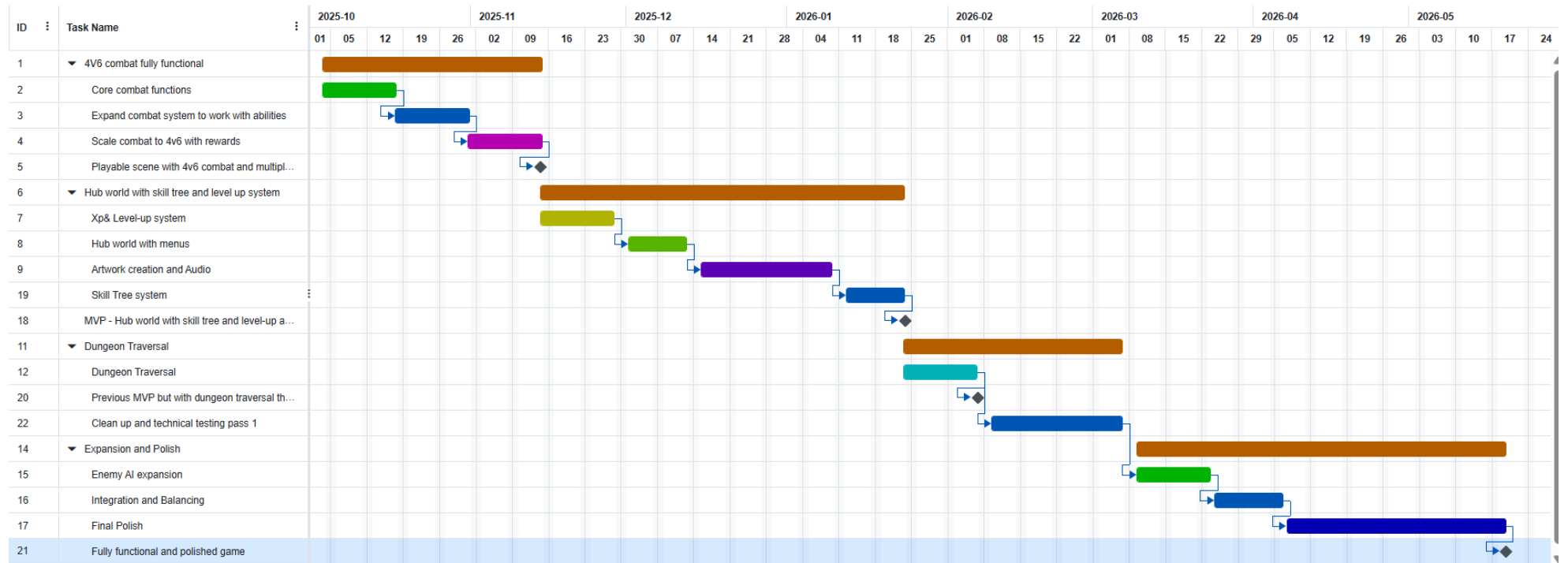


Figure 2 - Final Gantt chart

Sprint Backlog

+ Add a card

Sprint 1 - Core Combat Foundations

✓ Set up GameManager (singleton, persistent).

✓ Create CharacterData + EnemyData (ScriptableObject).

✓ Runtime CharacterInstance + EnemyInstance

✓ CombatManager: 4v1 flow.

✓ Turn Manager

✓ MVVM-style HP/UI bars.

✓ Implement hidden buttons for character order selection

✓ Implement end-turn cycle across 4 characters

✓ AudioManager singleton + placeholder SFX.

✓ Deliverable: 4V1 combat system with character and target selection with 1 basic attack and end turn buttons.

+ Add a card

Sprint 2 - Expand Combat system with abilities

✓ Add cooldowns, buffs/debuffs, skill targeting.

✓ Implement system for timed buffs/ debuffs

✓ Expand to 4v2 combat.

✓ Create new Enemy type with unique ability and different stats

✓ UI: skill cooldowns

✓ Deliverable: 4V2 combat system but with ability buttons instead of a basic attack, cooldowns and proper targeting for each skill.

+ Add a card

Sprint 3 - Scale to 4V6 combat

✓ Make robust basic Enemy AI script to handle attack choices

✓ UI: enemy/character health bars in world

✓ Add multi-target abilities

✓ Add new ability types

13/13

✓ Have intelligence stat effects and have bool to select if it effects that ability

✓ Add magic defence and luck stat and affect appropriate systems

✓ Test all new ability types

✓ Fix highlight for selected character and add new UI to show Target(s)

✓ Add passive abilities

2/2

✓ Update UI to handle enemies

+ Add a card

Sprint 4 - XP & Progression

✓ Gain XP after battle is complete - character to kill enemy gets majority and the rest of the party also gets some

✓ change required xp per level

✓ character level-up UI

✓ Stat increase for level up and player choosing stat increases

✓ Minus signs show up when player has assigned skill point to reverse

✓ ensure XP and levels persist between sessions

✓ every x level increase class of character e.g. F→S

✓ Add win/loss screen with xp gained for each character and leveled up text

✓ Advanced ability types

11/11

+ Add a card

Sprint 5 - Hub world implementation

✓ add interactable objects for gameplay features (e.g. shop, quest board, dungeon gate)

✓ Make interactive objects display placeholder ui

✓ Implement shop upgrade functionality

Implement training functionality

✓ Add dungeon selection board ui and functionality to start combat

✓ Add partial functionality to skills menu - stats switching for each character and stat allocation screen

✓ Deliverable: A functional hub scene where the player can move between and interact

+ Add a card

Sprint 6 & 7 - artwork, animation and fx

✓ Make advanced icons

6/6

✓ Artwork for characters

4/4

✓ Artwork for enemies

2/3

✓ Idle animations for characters

4/4

✓ Idle animations for enemies

2/3

✓ Death state (alpha reduced)

✓ Effects for hits

✓ Fx for buff

✓ Fx for debuff

✓ Fx for stun and sleep

✓ Fx for shield

+ Add a card

Inbox

Planner

Board

Switch boards

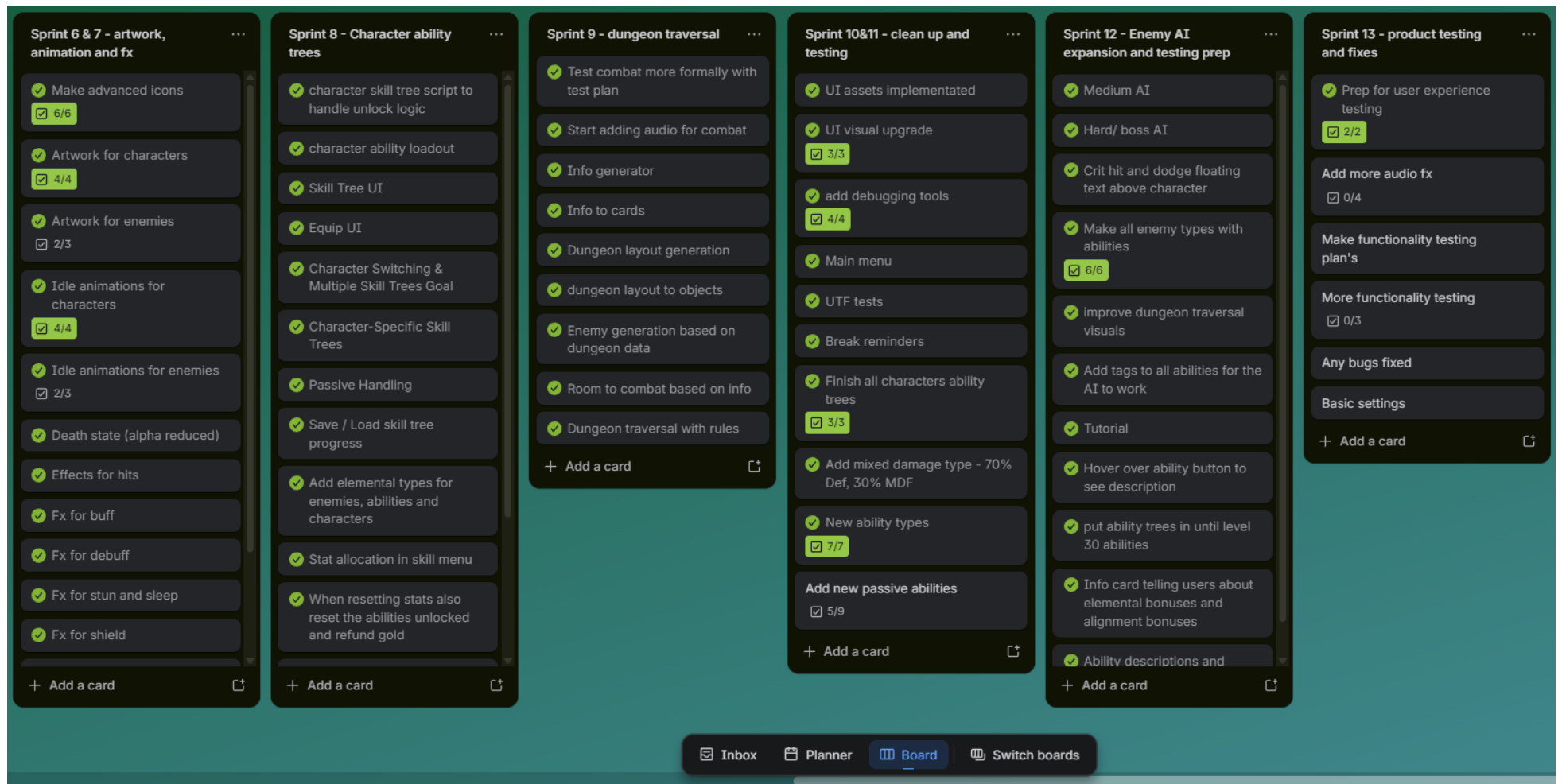


Figure 3 - Sprint backlog (Trello board)

5 Methodology

Before any development began a stringent planning process was undertaken. This first consisted of doing some preliminary research to find appropriate structure the game could have. Then a theme and some of the main mechanics were explored and the best suited for the aim was picked. Some alternative methods were considered before implementation began.

5.1 Alternatives considered

A puzzle game was considered instead of a combat game as puzzles allow people to learn a concept quicker as they need to understand the problem in depth and what makes the solution work. However, for strategic skills, designing a game with a low barrier of entry but with a high skill ceiling was difficult. So that the game is accessible and challenging. Creating a new concept for a game or puzzle like this proved to be an immense challenge while also standing out from current solutions. Some people also find puzzle-style games like this boring and therefore less engaging. I needed a solution that could be engaging for as many of the target audience as possible. Real-time combat was also considered for this problem as it is more demanding on the players which would increase the capacity of learning potential. However, designing a system that ensures players are creating and using strategies rather than being in a routine of gameplay and just progressing with upgrades would take a lot of resources. This would diminish the planning and evaluation of strategies leading to less of the target skills being developed. Real-time combat being more demanding of a player is also a downside as the target audience (10–14-year-olds) may not have much experience with playing games where a lot of the other skills required are developed. Real-time combat is also more cognitively loaded and could harm learning outcomes if it isn't balanced correctly. This may make the product less attractive to people new to video games as progression may be slower or the game may be too difficult for them early on. A turn-based approach reduces the number of controls as people new to games may not be comfortable using a keyboard for quick inputs whereas taking their time clicking buttons with a mouse is easier. The nature of a game being turn-based allows for time for players to plan, evaluate and test strategies. Therefore, a turn-based game was decided upon as it supports reflective thinking and experimental learning.

5.2 Development approach

For this project I used an agile methodology with a sprint-based structure. I mainly chose this structure as it would let me get feedback on my progress and any design decisions regularly while also keeping to a set time schedule. This schedule could be adjusted for sprints overrunning or if I allocated too much time for the development of something. I followed a two-week structure for a sprint, with thirteen in total, where at the end of one I would plan out the specific systems to be developed. During this time, I would assess if the main sprint goal for the current sprint was met. If it was not some additional work would be added to the next sprint to ensure I didn't fall behind schedule. Some major systems were split over multiple sprints such as the combat system. This was split over 3 sprints with each sprint developing a different aspect or expanding on what was already built. For example, I started with the basic flow of combat with a basic attack button and no targeting with one character and enemy, then I expanded the system to include targeting and choosing a character and finally adding the abilities and multi-targeting. After a sprint meeting with my supervisor, I would reflect on the feedback and decide if any of the features or changes should be implemented and when it should be done.

Sprint	Feature	Outcome
1	Combat base	Turn flow working
2	Abilities	Player choice added
3	Multi-target	Strategy depth increased

I also used iterative development for this project as this is a very design heavy game. Using iterative development let me try new approaches and see how they would affect the project quickly which allowed me to try out many more options than traditional development. The sprints for the whole project were planned in the initial stages with some time at the end allocated in case of overrun or for extra components to be implemented. I regularly tested the project informally, through play testing, when a new feature was added with any logic errors not relating to the new content being added to the bug log to be fixed later. The play testing was considered a success if its behaviour was as designed in a variety of contexts. This let me gain a large amount of functionality quickly while it is working well enough for a demo. I then had a few formal testing sprints throughout so that I could spot any bugs that were less vital to the project. This consisted of making testing plans that were designed to find bugs not just see if the system worked as intended. Sprint reviews helped to inform my design decisions for the next sprint to be undertaken and help me consider options I hadn't before.

5.3 Evolution of requirements

The requirements of this project evolved mainly through the planning process. This was mainly done to reduce the scope to the key functionality to ensure a complete product was produced. In the initial stages of the planning process some small puzzle rooms were included to test the players in other areas such as memory. However, it was decided that this wasn't completely necessary to the main goal of this project as it didn't test the main skills to be targeted and was added as a stretch goal instead. The combat initially was designed to have some movement involved in a grid with attacks also being grid based. However, I believed this might give players too much to learn early on and could overwhelm them. So, it was decided a more focused approach on the combat and core abilities was necessary. The benefits of the added depth to combat with movement was evaluated and decided not to be worth the extra cognitive load.

During development, some aspects of the project also changed, for example initially it was planned for 3 main stats of attack, defence, and health points. This evolved during development as there wasn't enough depth for the combat to match the aim of the project. I added an intelligence, magic defence, and luck stat. The intelligence and magic defence were used to scale different damage types and for the use of chances of effects being applied. Luck was used to add some randomness to the combat with critical hits and dodging, where with a higher luck stat, the higher chance of them occurring. This improved the strategic depth as players couldn't predict the course of combat and would have to react to the combat as it occurred. It also increased the possibilities of strategies the players could make adding more depth to the possible strategies.

5.4 Feedback incorporation

Throughout this project feedback from stand-up meetings were incorporated into the project. To do this I decided if the feedback and changes I could possibly make improved the project by comparing what the changes did compared to the aims. If it did, I then made a short plan on what I would change and roughly how to implement it. If the time required fit into the next sprint I would incorporate it at the end of the sprint. However, if it would take longer to implement then I would either wait for a time when I had a less busy sprint or added it as a stretch goal. This was decided on the predicted impact of the new feature and the risk it would present to falling behind the development schedule. One example of this was with the element system for the combat. My supervisor suggested this but only as advantages and disadvantages between different elements. I decided to expand this during the small planning phase to include affinities for abilities and a general boost. This was then implemented during the ability tree development sprint as I finished before the sprint ended. It was decided to be implemented due to the perceived benefits to the strategic depth by adding new variables the players needed to think about in the planning and evaluation phases.

6 Implementation

6.1 Architecture

The system architecture as shown in figure 4 was used to create this project and represents the flow of data between systems. To manage the games flow and scene switching I decided to use a singleton called GameManager to handle this. I decided on a singleton as it could be accessed from any scene by any script so I could more easily handle the flow of data. Being a singleton means that only one instance of it can exist which would reduce the errors associated with having multiple game managers such as using data from the wrong manager. However, a singleton can cause issues in larger projects when testing and reduces the maintainability. As this is a small to medium project this should be fine. The game manager is responsible for starting and stopping the combat when the combat system tells it the combat scenario is over. This is done by producing the combat data such as the enemies and preparing the character's data to be used. The combat system is made up of a combat manager, turn manager and enemy turn manager. This handled the flow of the combat and to check when a win or loss occurred. The combat system then connects to the UI and character system which allows the player to pick an ability to use. The available abilities are set in the character system. The character system also holds the character stats which are used to calculate damage. The enemies also use the character system for this purpose with the difference that they are not player controlled. The progression system is a simple XP based progression system which calls the UI system at the end of combat to allow the player to increase the stats of a character. The AI system defines the enemy's decisions they would make in the combat scenario. Finally, the ability system defines an ability's behaviour as well as the metrics used in the combat scenario such as the cooldown and uses. This architecture was designed to maximize modularity and increase capacity for iterative development by allowing new systems to be added efficiently.

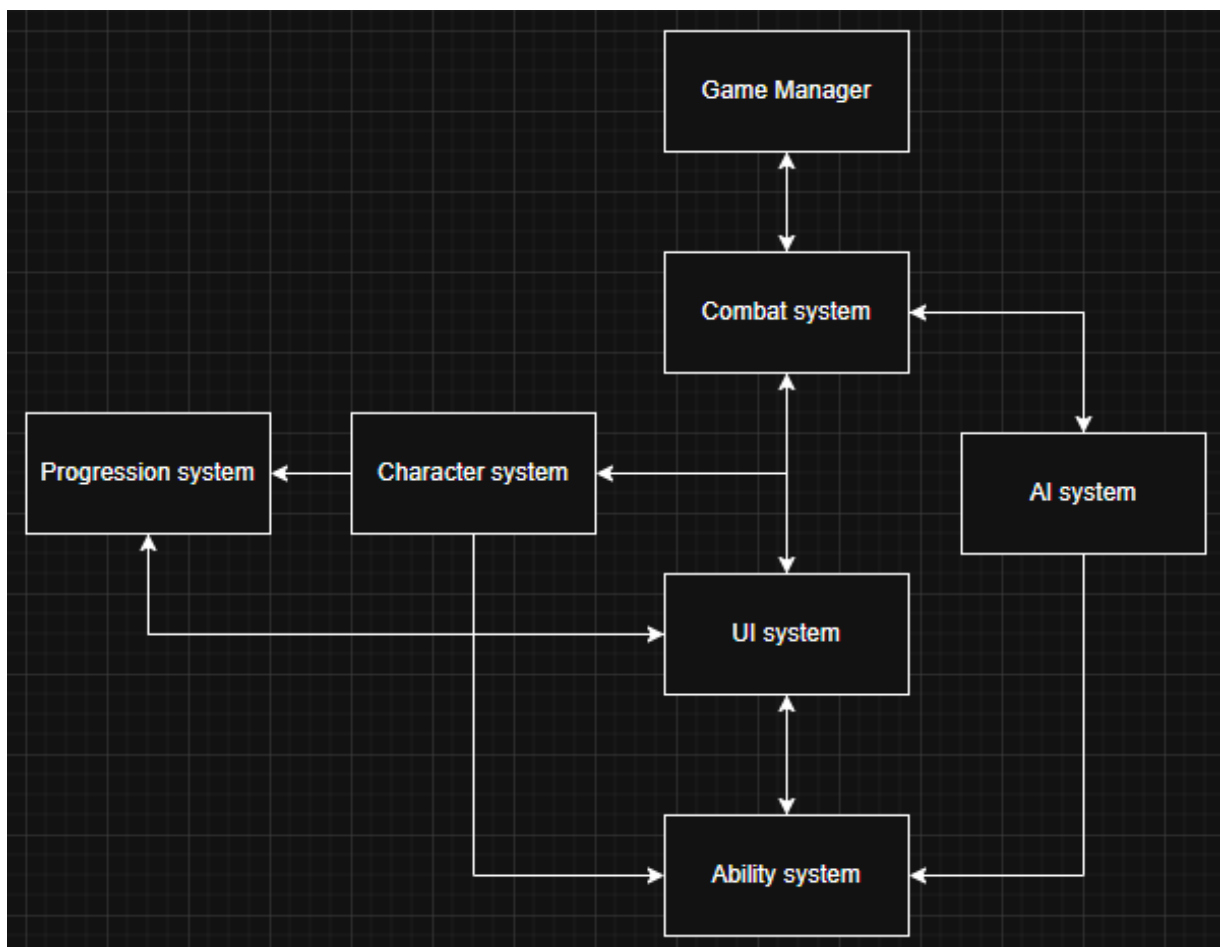


Figure 4 - High-level system architecture design

For this project I decided to use multiple scenes such as main menu, hub world, dungeon traversal, and combat. This allowed me to separate the logic for each section of the game. This meant that development was easier due to the different sections not interfering with one another. The only overarching system was the game manager. Using different scenes like this also allowed me to test features quicker as I knew a new feature wouldn't be affected by another from a different section. This style of implementation also reduced integration issues as there was only one point of contact between the scenes. This also supported the agile iteration as a feature in one scene can be added or changed with minimal need for integration to the overall system.

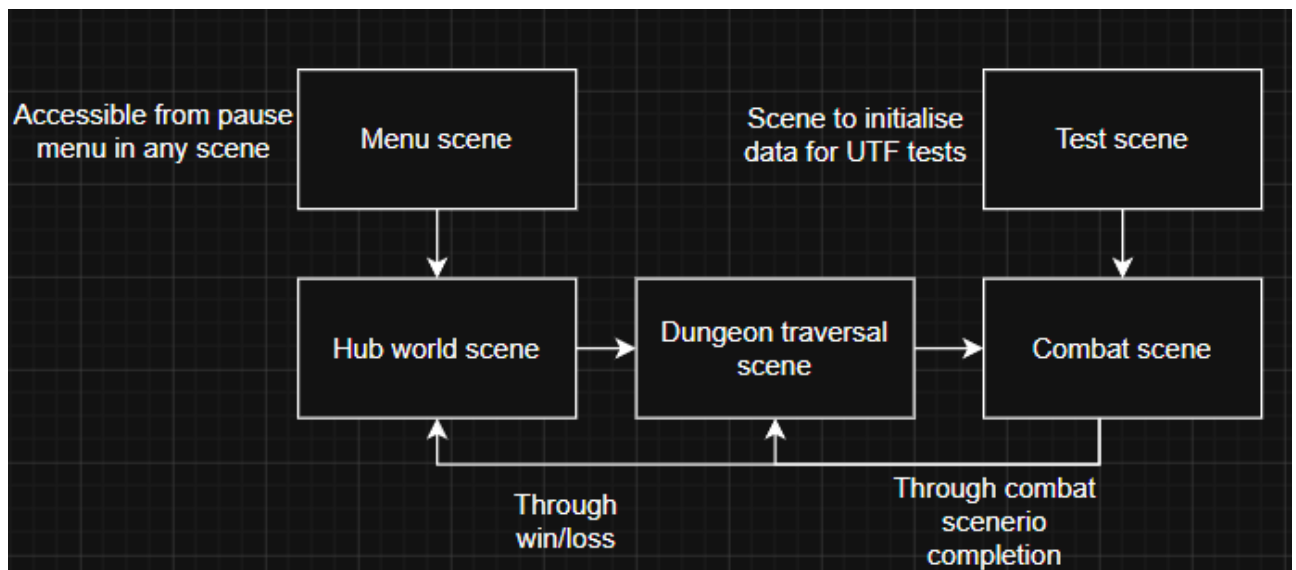


Figure 5 - Unity scene hierarchy and flow

6.2 Core systems

This system was implemented using a turn-based structure to support reflective decision making as established in the literature review. For the main combat loop a few managing scripts are used to manage the players actions and when they can perform them, the turn ordering logic, and the enemy AI manager to enable the AI to have a turn following the correct logic. The combat system was implemented over three sprints. Starting with the core combat foundations being the basic turn-taking with four vs one, then moving onto ability selection and single targeting to finally expanding the combat to four vs six and having multiple targeting abilities. These were called combat manager, turn manager and enemy turn manager, respectively. The combat manager handles the UI interaction for the player by loading the correct UI and linking the correct data. The combat manager is then responsible for starting and ending the combat loop and passing the data from the UI to the turn manager. The turn manager handles the logic of the player and enemy phase. This involves individual character and enemy turns, phase changes, and win-loss conditions. The turn manager calls the correct events to let the combat manager know to change the UI. The turn manager also holds the data for the current turn being picked which includes the character selected, ability used and the targets selected. The UI uses an MVVM (model, view, view-model) to manage the UI and data assigned to it while keeping the logic separate to reduce complexity. A visual style for controls was used so that it is more accessible to more players and reduces cognitive load for players unfamiliar with using a keyboard. The enemy turn manager goes through the enemy list and calls the enemy AI script attached to generate a valid turn. This is due to there being different levels of AI intelligence. Once this is done the enemy turn manager passes the turns to the turn manager to execute. The system was split into 3 as the distinct phases required different logic and having this separation allowed for the turn managers to be developed individually. Then the combat manager would supervise the flow of data from the combat to other systems. This structure helped the turn-based combat's clarity as there was a clear separation between the player and enemy phases. This implementation provided a low-risk environment to experiment with new strategies.



Figure 6 - Combat scenario UI

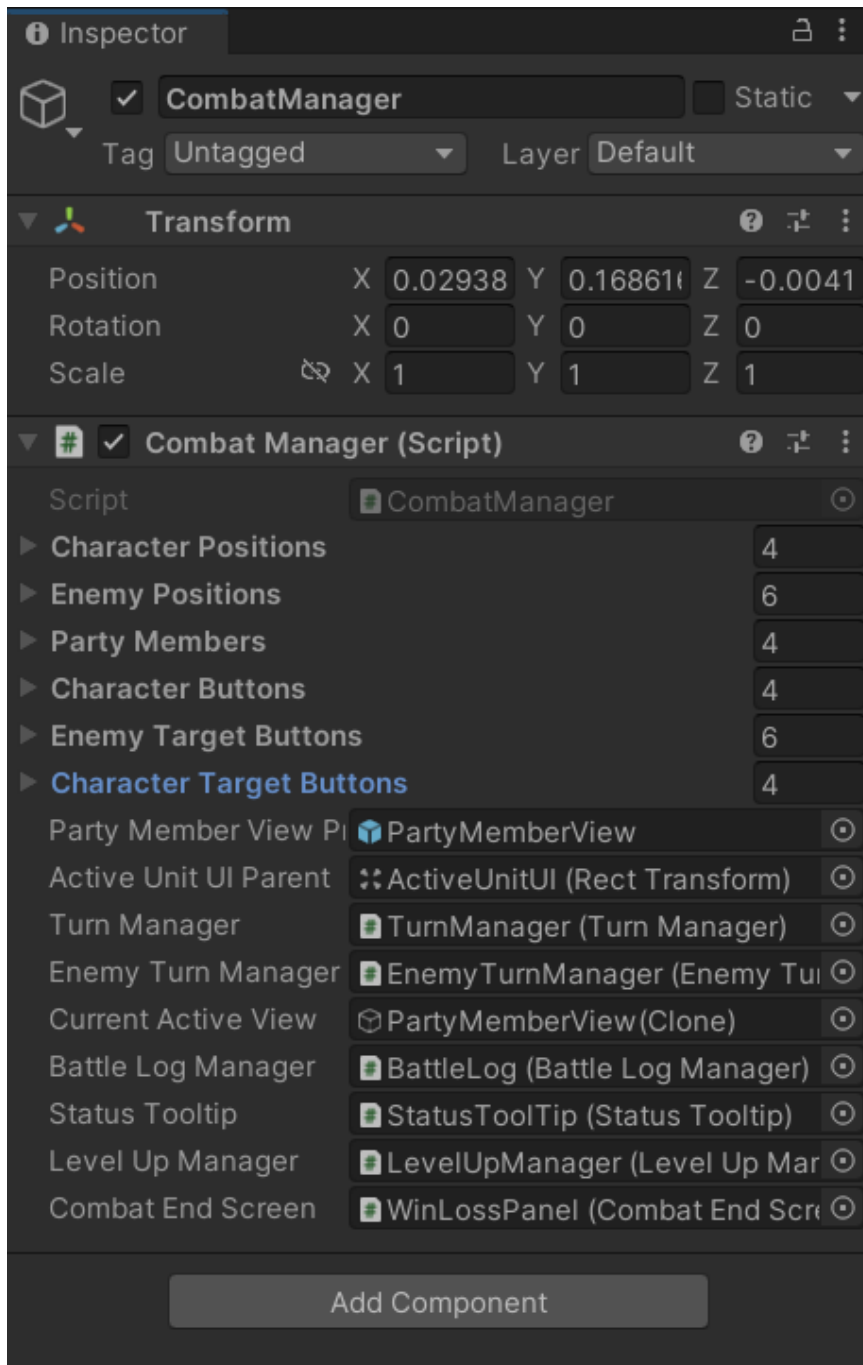


Figure 7 - Combat manager implementation

The ability and effects system was designed to be extremely modular so that new ones could be easily added. This was developed throughout the project starting at sprint two with basic abilities for attacking, buffs and debuffs. Then different ability types were added until passive abilities were added and having a physical and effect behaviour. Finally, the element system for abilities was added, this will be discussed in more detail later. This was done using scriptable objects (SO) with the logic then attached. Each logic function was a child of the overarching ability function so that they all had the same virtual execute function. To add a new ability behaviour a new class had to be defined, and the logic written then it could be used after attaching it to an ability SO. The effects were made in a similar way so that new ones could be quickly added except for a class as a child of an overarching effect class. This is done so managing the effects and applying them to an entity is a lot simpler and robust. This supports scalability as new abilities can be added quickly through the editor and new ability types by adding a new behaviour script with the

logic. This reduces development time as abilities with attack logic and an effect can be combined instead of having to make a new script with both included. This also supports the agile method as a developer can quickly add a new ability mid-sprint with minimal development time. Using this structure also helped add strategic depth to the game as new ability types could be added to expand the possible strategies used.

The image shows the Unity Inspector window for an ability named 'Invigorate'. The window is titled 'Inspector' and 'Invigorate (Ability Data)'. It contains a form with various fields for configuring the ability. The fields are organized into sections: Script, Ability Name, Description, Icon, Ability Category, Power Type, Ability Type, Tags, Max Usage, Cooldown, Target Type, Number Of Targets, Physical Behaviour, Effect Behaviour, Passive Behaviour, Guaranteed Effect Hit, Effect Chance Scaling, Effect Icon, Is Taunable, Element, Is Element, Element Tag, Boost, Ability tree data, Unlocked, Prerequisite Abilities, Strength Required, Defense Required, Intelligence Required, Magic Defense Required, Luck Required, Vitality Required, Gold Cost, Tree Col, and Tree Row.

Field	Value
Script	AbilityData
Ability Name	Invigorate
Description	Increases strength by 25-45% for 3 rounds. [targets 2] [Uses 30] [Cooldown 3]
Icon	skill_icons15
Ability Category	Active
Power Type	Physical
Ability Type	Buff
Tags	1
Max Usage	30
Cooldown	3
Target Type	Multiple Ally
Number Of Targets	2
Physical Behaviour	None (Ability Behaviour)
Effect Behaviour	STRbuff_3_0.25_0.45_0.6 (Buff Behav)
Passive Behaviour	None (Passive Behaviour)
Guaranteed Effect Hit	☒
Effect Chance Scaling	0
Effect Icon	None (Sprite)
Is Taunable	☐
Element	
Is Element	☐
Element Tag	None
Boost	None (Element_boost)
Ability tree data	
Unlocked	☐
Prerequisite Abilities	1
Strength Required	21
Defense Required	21
Intelligence Required	48
Magic Defense Required	26
Luck Required	33
Vitality Required	26
Gold Cost	0
Tree Col	6
Tree Row	0

Figure 8 - Ability SO example

```

[CreateAssetMenu(menuName = "Abilities/Behaviours/Buf")]
public class BuffBehaviour : AbilityBehaviour
{
    public List<StatType> statsToBuff;
    public int durationTurns;

    public float basePercent;
    public float maxPercent;
    public float intelligenceScaling;

    public override void Execute(CombatMember user, List<CombatMember> targets, AbilityData ability)
    {
        foreach (var target in targets)
        {
            float scaledPercentage = target.CurrentIntelligence * intelligenceScaling + basePercent;

            if (scaledPercentage >= maxPercent)
            {
                if (Random.Range(0, 100) <= target.GetEffectApplyChance(user, ability)) // random chance to apply for certain abilities
                {
                    foreach (StatType statToBuff in statsToBuff) // for multi-stat buffs
                    {
                        BuffEffect effectToAdd;
                        if (user.element == ability.elementTag && ability.elementTag != Element.None && ability.boost != null) // checking for elemental synergy
                        {
                            effectToAdd = new BuffEffect(statToBuff, maxPercent * ability.boost.multipliedPotency,
                                durationTurns + ability.boost.addedDuration);
                        }
                        else
                        {
                            effectToAdd = new BuffEffect(statToBuff, maxPercent, durationTurns);
                        }
                        effectToAdd.name = ability.abilityName; // setting data for the tooltip on effect icon
                        effectToAdd.description = ability.description;
                        effectToAdd.icon = ability.EffectIcon;
                        effectToAdd.colorType = colorType.Positive;
                        user.ContributionPoints += 0.5f;
                        target.ApplyEffect(effectToAdd, false);
                    }
                    target.SpawnBuffEffect(); // spawns particle FX on character
                }
            }
        }
    }
}

```

Figure 9 - Ability behaviour code example (Buff effect)

Each character has different stats the player can upgrade when the character levels up. These stats are used to unlock new abilities and calculate damage received from an ability and scale the damage of an ability. These damage calculations are done using mathematical equations to ensure that progression is fair with the levels involved. This is the equation for the damage calculations $1 - \max DR \times (1 - e^{-D/kd})$ in this equation $\max DR$ is the maximum damage reduction which is used to ensure that even with large stat differences some damage is applied. The kd is the scaling constant which can be adjusted to change the damage for each level difference. Finally, D is the defensive stat associated with the damage type for example a magic attack would use magic defence. This is then multiplied by the ability damage scaled to the associated stat to get the final damage received. This was chosen to ensure there was fairness between stats by having a soft maximum of damage that can be dealt, that is why a linear equation was not chosen. If there was too much of a difference the player could die instantly. This reduces the effect of stacking too much into single attributes and rewards a more balanced approach.

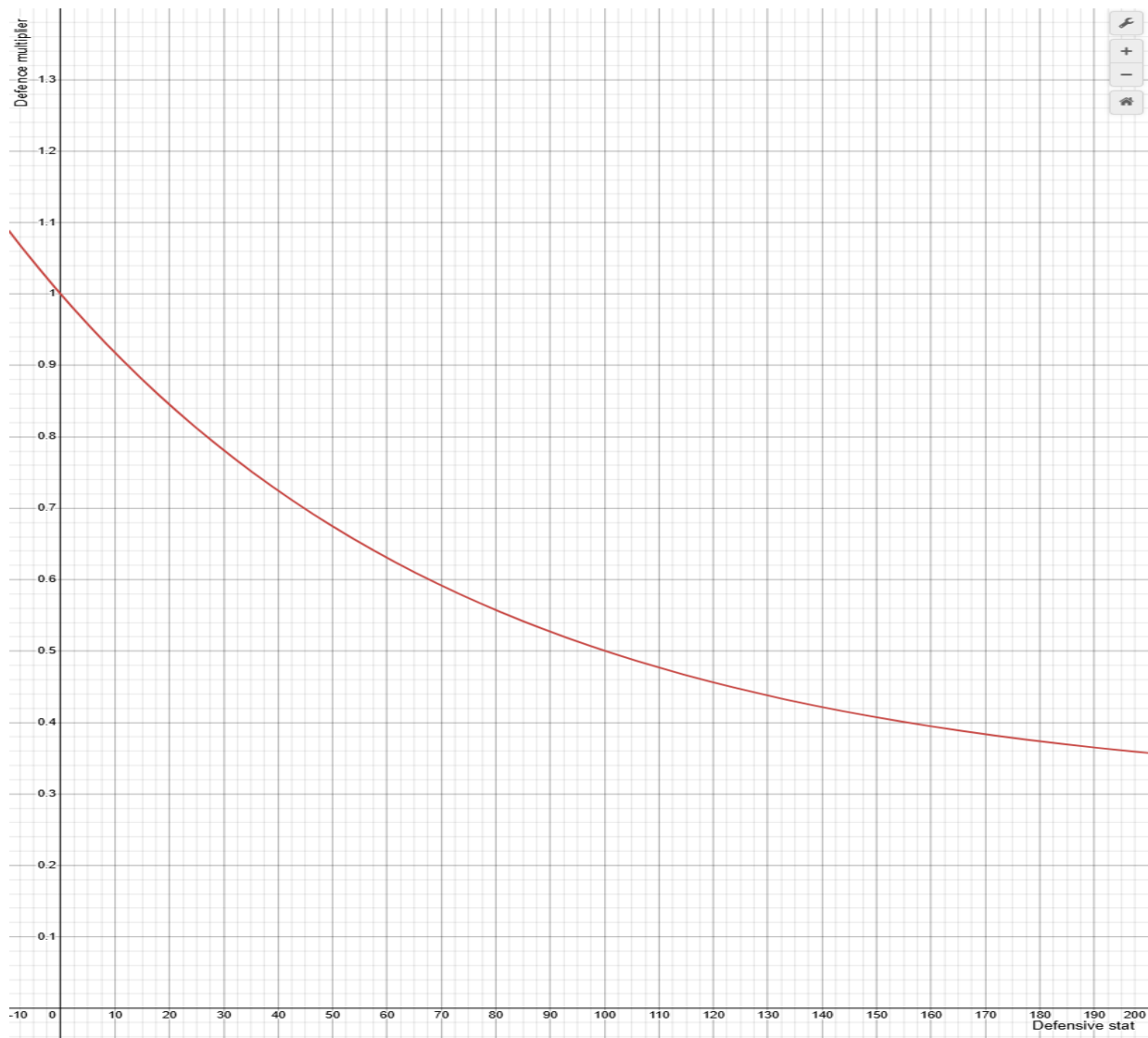


Figure 10 - Damage calculation scaling graph

The skill trees work by linking a SO to a UI component that can be dragged and dropped to let the player equip an ability. I decided to use a UI component so that I could have all the functionality needed such as the drag and drop but also the player being able to click the ability to see more details and unlock it. The skill tree is built off a list of abilities that have the row, column, unlock requirements which include the stats needed and prerequisite abilities. This is passed to a skill tree manager script that builds the skill tree using prefab objects for the buttons and links the ability to the skill button script. This is done in order so that all the abilities are in the correct places and linked correctly too. Finally, the links are added visually with the colour depending on the unlock state of both abilities. Implementing the skill trees this way rather than making them by hand allows me to edit the skill trees easily and quickly so I can iterate through skill tree designs for each character. It also makes the system very modular as I just need to add abilities to the list attached to the character and they will be added to the skill tree UI. This allows a developer to quickly and easily edit a skill tree by simply changing the SO values. The skill tree having stat requirements for unlock means that players can't rush to the higher tier abilities. This also supports the players long-term planning (Bottino et al. (2007)) as they must ensure that they invest in the correct stats to unlock abilities they want to use. This also gives players many abilities to experiment with, this will support their learning further Gouveia et al. (2011).



For the dungeon traversal I used a simple random walk to generate the grid layout of the dungeon. I chose a random walk as the layout didn't need to be complex and I needed an efficient way to ensure all rooms were connected. The data for the dungeon such as the length, difficulty and rewards is generated separately to then be used in the generation of the layout and for the information about a dungeon. The player traverses the dungeon by selecting an adjacent room to the one they are currently in. Each room is either a combat scenario or a small reward. The game manager holds the dungeon data that was selected by the player which is then used to generate the layout then converted to game objects inside the dungeon traversal scene. In figure 11 the semi-transparent rooms are ones the player can visit but hasn't yet. The fully opaque rooms are ones they have visited there is not current room they are in for this implementation. Finally, rooms that they can't reach are fully transparent to ensure they explore and not just head straight to the boss room. Using randomness was decided upon to improve the variation in the learning experience and to prevent memorisation so that players make robust strategies not ones specific to a level.



Figure 11 - Dungeon generation example

For the characters, ability data and ability behaviour scriptable objects (SO) were used to follow data-oriented design. This allows for one script for each to be made then an instance of the script to be created where all the parameters can be defined. This is very useful for duplicating logic while still having separate data. The character SO hold the stats for each character as well as any information needed for the UI such as the name or the icon. The same character SO is used for the enemies as they are functionally the same for the objects. The ability data SO holds information about the ability such as the cooldown time and uses. However as SO are persistent over instances of gameplay this data is copied to a runtime script to ensure that the original data is preserved. This also means the same SO can be used on multiple characters without the ability data being changed by multiple. This was an issue before I converted them to a runtime scripts as one character would use the ability and the cooldown would not allow another to use it. This meant that some abilities were unusable as the enemies would use them and be on cooldown ruining the players planning. This then increases the scalability of the project as only variations of an ability need to be made not duplicates. The ability behaviour SO holds the logic of the ability with some data values to allow for differentiation. Mainly the behaviour SO applies effects to the character which can be a chance or guaranteed hit. This means that in the ability data SO there are multiple slots for the ability

behaviour one for a primary and one for an effect. The same ability data SO is also used for passives so that the same framework for application of the ability can be used.

For this project three levels of classical game enemy AI were used and took one sprint to implement. The various levels of AI are meant to simulate what the progression of a player's strategy may be and to challenge them with various depths of strategy to counter. This increase in complexity also challenges the players more, leading to more learning as discussed shown in Bottino et al. (2007) and to challenge strategic adaptations to new encounters as supported in Glass et al. (2013). Firstly, the easiest AI was just a random pick of an ability and a valid target. This has no real strategy or value linked to what ability is best to pick. The middle difficulty AI uses a utility-based ability selection with a heuristic targeting logic. Each ability is scored on various categories, and the highest scoring is picked. Then the best target is selected based on which the ability will have the best effect on. For example, for a damage ability it will target the lowest HP enemy. This gives the AI a slight strategy of targeting the weakest first or buffing the strongest of the team. Finally, the hardest AI uses full utility-based decision making. This AI evaluates the ability and target combinations and selects the highest scoring. This means that the AI evaluates the complete action rather than just abilities. This is done similarly to the medium difficulty AI but is looped through all pairs. Some additional logic is added such as more score for killing an enemy to increase the difficulty further. For the medium and hard AI, a random score is also added so that the AI doesn't make perfect decisions constantly which will give players a chance to take the advantage when they spot a mistake.

7 Testing & Evaluation

7.1 Testing strategies

For testing the core functionality such as checking for a win or loss and damage calculations I set up automated testing using Unity's testing framework (UTF). This allowed me to ensure that when I implemented features that could interfere with the core functionality I could quickly test and get a definitive answer to check if they still worked as required. This saved a lot of time as it takes minimal time to run. When setting up the UTF tests I added various null checks which I previously didn't have. These will help reduce the chance of the product crashing due to an unknown error.

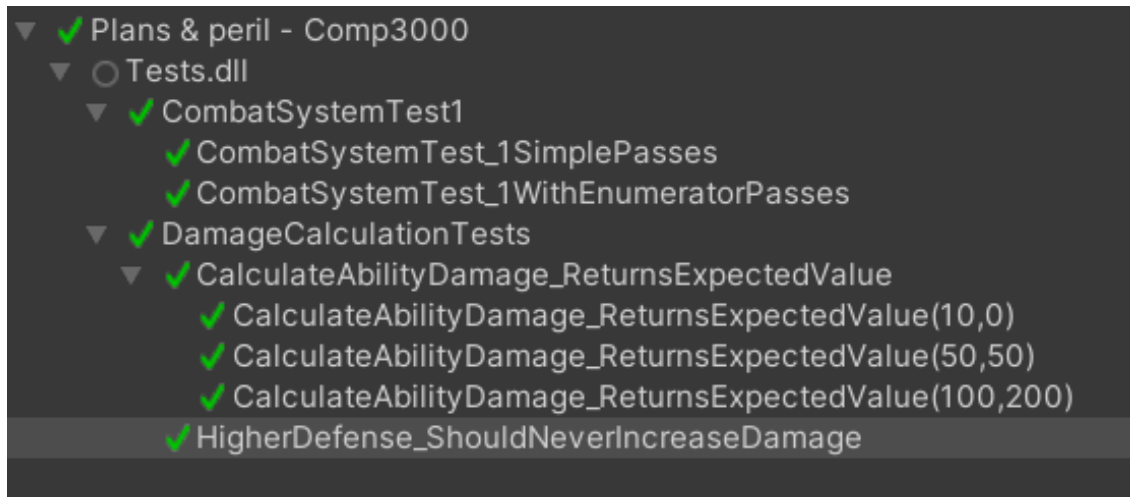


Figure 12 - UTF test results

```

references
public class DamageCalculationTests
{
    CombatMember attacker;
    CombatMember target;
    AbilityData ability;

    [SetUp]
    Tabnine | 0 references
    public void Setup()
    {
        var attackerGO = new GameObject();
        attacker = attackerGO.AddComponent<TestCombatMember>();

        var targetGO = new GameObject();
        target = targetGO.AddComponent<TestCombatMember>();

        ability = CreateBasicAttack();

        attacker.CurrentAttack = 0;
        target.CurrentDefense = 0;
    }

    [TestCase(10, 0)]
    [TestCase(50, 50)]
    [TestCase(100, 200)]
    Tabnine | 0 references
    public void CalculateAbilityDamage_ReturnsExpectedValue(int attack, int defense)
    {
        attacker.CurrentAttack = attack;
        target.CurrentDefense = defense;

        float maxDR = 0.7f;
        float kd = 80f;
        float baseDamage = 100f;

        float raw = baseDamage * (1f + ((float)attack / 100f));
        float expected = raw * (1f - maxDR * (1f - Mathf.Exp(-((float)defense / kd))));

        float actual = attacker.CalculateAbilityDamage(attacker, target, ability);

        Assert.AreEqual(expected, actual, 0.01f);
    }

    [Test]
    Tabnine | 0 references
    public void HigherDefense_ShouldNeverIncreaseDamage()
    {
        attacker.CurrentAttack = 50;

        target.CurrentDefense = 10;
        float lowDefDamage = attacker.CalculateAbilityDamage(attacker, target, ability);

        target.CurrentDefense = 200;
        float highDefDamage = attacker.CalculateAbilityDamage(attacker, target, ability);

        Assert.Less(highDefDamage, lowDefDamage);
    }
}

```

Figure 13 - UTF test example

As I was implementing other features and came across a bug not related to the feature I was currently working on, I would add it to a bug log I created to keep track of bugs, their severity and what the fix was. The severity level ranged from game breaking to a visual problem. I used the severity level to know how soon I should fix a bug as if it was a game breaking one it could interfere with other features that need implementation. I didn't include bugs on features I was currently developing as I could fix those quickly

and I saw them as part of the development process. The bug log acted as a backlog of bugs so that they were not forgotten and left unfixed.

The bug log was also leveraged during play testing. If any bugs were found during play testing, they were also added to the bug log for future fixes. Play testing sessions were conducted by me and were used to determine the fairness of the gameplay and adjust any of the equations used for damage calculations or enemy scaling. During my play tests I found I gave the player-controlled characters too little health to survive all the engagements and increased accordingly. Play-testing was also used to determine if the ability logic worked as intended which was determined through observation and a debugging overlay which displayed the current state of all characters and enemies and an expanded version of the combat log. This held data for the damage given and received as well as which entities were targeted. The unity inspector was also used to monitor the runtime data to ensure the UI matched the data correctly and updated at the correct time. During this the performance of the machine being used was monitored to ensure no frame drops or data leaks were occurring. This was done using the unity stats when running the game and found even on lower end devices the game was stable and smooth.

Date encountered	Description of bug	Severity level	Fix details	Fix date
03/11/25	When there is a single character left, and they have their turn it does not change to enemy turn. No inputs left. Tested further shows that if any character dies and all others have turn phase is not switched	4	Added code below to <code>TurnManager.ConfirmAction()</code> <pre>List<PartyMember> AlivePartyMembers = new List<PartyMember>(); foreach (PartyMember mem in PartyMembers) { if (mem != null && mem.Alive) { AlivePartyMembers.Add(mem); } } if (AlivePartyMembers.All(m => !m.HasTurn)) ExecutePlayerActions();</pre>	03/11/2025
05/11/25	When using multi-target abilities on enemies no target is selected on click	3	Two pieces of code were adding to the list of targets the first would add to the list the second would then remove. Changed <code>OnClick()</code> in <code>TargetSelectButton.cs</code> and removed call of <code>//TurnManager.PlayerSelectedTarget(Enemy);</code> <code>PlayerSelectedTarget</code> is now only called once.	05/11/2025
07/11/25	Effects tick twice per round for enemies	2	Removed duplicate of <code>EndEnemyPhase();</code> left from before <code>EnemyTurnManager()</code> was implemented	07/11/2025
07/11/25	Enemies couldn't apply buffs to their allies. Due to <code>GameManage.Instance.EnemyMembers</code> being empty on start of combat.	3	Refreshed the list at the start of <code>Turnmanager</code> .	07/11/2025
18/11/25	When using abilities that target all enemies the targets do not get reset when a new ability is pressed and can still confirm. So you can select an ability meant	3	Added lines: <code>currentActiveView?.GetComponent<PartyMemberView>().HideConfirmButton();</code> <code>turnManager.SelectedTarget.Clear();</code> to all target button showing functions	19/11/2025

Figure 14 - Bug log extract

After finishing development of a feature, I would informally test it by using it in gameplay and seeing if it acted as I wanted it to and spot any bugs. This was iterated until I could play multiple instances with no bugs occurring or behaviour that was unexpected. To test a feature informally I would start by using it as I intended it to be used, this meant not trying to break it in any way and see if it worked smoothly. Once that was achieved, I would start the process of trying to use it incorrectly through sequence breaking, using incorrect inputs, or finding edge cases. This process let me find most issues relating to the new feature and let me quickly polish it without spending a great deal of time on formal testing. This meant that features were ready quicker which let me get feedback or start another sooner. However, this was not

the only testing done as I planned to have a formal functional testing session to ensure that no bugs were missed and the product ran smoothly.

During the formal functional testing the main game systems were tested as testing all features was not possible due to time constraints. These tests were to validate that the game's main systems worked and that a player could actually play. The play testing was used to find smaller errors that needed to be fixed and were mainly UI oriented. Through this testing I found my save and load system wasn't functioning and added a save manager that handled saving and loading core data.

7.2 User experience testing

To test the products accessibility to inexperienced users and enjoyment I assessed it with user experience testing. I chose these metrics as they are important for strategic skill development as discussed in the literature review. I used volunteers that are completely unfamiliar with the product and systems developed. A session consisted of the user completing the tutorial then playing one round of combat to get a feel for the gameplay then completing a short survey to assess their experience in accessibility and enjoyment. This resulted in some very helpful feedback to base improvements off and some observations of how the player interacted with the UI. However, the sample size was too small at 5 participants so further testing is required to confirm these results or additional testing after initial changes. The number of participants should be enough for initial feedback to improve on before a second round of testing with new users. On average the users rated the clarity of the game at a 4/5 the ease of use at a 4.2/5 and the enjoyment a 4.4/5. This told me that the game was enjoyable, but the tutorial wasn't fully clear for the users to follow. This matched my observations as many users got stuck at some points during the tutorial requiring additional support. To remedy this, I added some new features to aid the tutorial. Many improvements were making certain UI features bigger as some were a bit too small and therefore not obvious enough. Button animations were also added to show the player where they needed to click next to aid them further and a visual combat order queue was added as some participants relayed confusion over the combat order as most other games use a priority system unlike this one where the player chooses the order. To confirm that these changes had a positive effect on the aspects that were tested a second round of user experience testing was undertaken.

For the second round of user testing the same methodology of the first round was used but with new subjects as to avoid any familiarity of the system. This round of testing showed promising results in the improvements identified after round one. However, this session only had 3 participants so the results may be due to this small pool of participants. The feedback given did not identify any major improvements. The majority of the improvements focused on clarifying the tutorial for the player. This was shown to have been achieved as the clarity score improved from an average of 4 to an average of 4.6. The enjoyment and ease of use scores didn't change significantly meaning that these changes didn't affect these aspects.

7.3 Discussion

I think my design and implementation of the ability system went very well as it allowed me to quickly and easily add new abilities for each character. As it was so scalable it also allowed me to add new ability types to have a wide range for the player to use. I think the effect system also was implemented well for the same reasons. After it was set up it was easy to add new logic for when it was applied, removed and to tick. I also believe the skill tree and stat system works well too as it provides plenty of choice for the player allowing for more strategic depth without being too overly complicated. The skill tree system is also very scalable as the game objects are built from a list of abilities, so it is simple to add new ones. Overall, I think the scalability of my product is its biggest strength and would allow for easy expansion in the future. This would add even more strategic depth through more complex abilities and allow for deeper planning by the player.

I think my enemy AI implementation could be more robust and the different levels could be differentiated more. This is because throughout the testing of this it was very hard to spot any logic errors the AI had and some, I did come across were from easy oversights such as allowing multi-target abilities to target the same character more than once. I also think that more regular play-testing from other participants would have helped to know where any design changes should've been made earlier in the development process. This would have especially helped to know if my systems were going in the right direction and may have made two rounds of user experience testing redundant as I would have found those issues earlier on.

The main trade-off I made was reducing the number of ability types. This was because some of them were quite complex and would have required a lot of time cumulatively to implement. This time was used instead on other features such as the tutorial as it was more important than expanding the ability types that would only be used in the late-game phase. These abilities were still designed just not implemented. Another trade-off was reducing the amount of original assets for the game. I initially planned to make all the assets myself but after producing the 4 characters and 2 enemies I realized how long it takes to properly develop sprites. Instead of producing them all I used third-party assets to ensure the product had good visuals while also following the art-style I wanted.

Some systems didn't integrate correctly when developing this product. For example, with the tutorial it sometimes wouldn't reset the player data fully before running. This led a user being stuck in the tutorial during testing and having to skip the remainder of it. This generally happened because some of the abilities were still equipped even though they weren't unlocked leading to players not being able to unlock them. Another major problem during the beginning of development was when the transition from simple single target abilities to multi-target abilities was being implemented. This led to a lot of targeting bugs that took a large chunk of development time to fix. This was due to too shallow of a design and planning for the implementation as I thought it would be simpler to change than it was. Another issue during play testing was the difficulty being too high for multiple encounters. As the player characters had similar health to the enemies it was very unfair in multiple encounters with no healing between. To fix this I increased the starting health of each character so that they can survive a whole dungeon and it is not impossible to finish.

From the user testing results I can see that the product is generally enjoyable. This would lead to more engagement from the user as they are more emotionally invested in the experience. From the literature review this would mean that the knowledge gain would increase so the gain of strategic skills would be better. As I have also used a pixel-art style and followed the PEGI guidelines the product is suitable for the target demographic to play. These show that the aims of this project have been fulfilled and that the product could be used in an educational setting.

8 Conclusion & Future Work

8.1 Conclusions

Overall, I succeeded in producing a scalable and professional solution to the problem. I also learnt a lot of new technologies and design philosophies such as MVVM and singletons. I believe the aims for this project were met as from the user experience testing subjects enjoy using the product and from the literature review there is strong evidence to suggest that this product will successfully teach users strategic skills.

Some key technical takeaways I have is that a system architecture plan is extremely important to ensure that a product is maintainable and scalable. I understand that it is important to save time in the future as it allows for quicker development and less debugging time. I also learnt the benefits of modular systems as they allow for quick implementation of many features while reducing the overall workload and time required. This worked extremely well for the ability system as I only needed one system to use the ability, and they could be reused for the ability tree saving more time. I understand the importance of segmentation across a complex system. This is because during development I used different scenes to separate the logic of the main systems. This helped as when developing a new feature, I only had to think about interactions with the current system not others not part of it but in the same space. This also let me build the complex systems with lots of simple layers as they would only interact with each other. During this project I understand the value of scope management. During the planning phase I cut a lot of features out that could have added some value to the project but may not have been completed in the time of development such as an item system. This was instead added as a stretch goal if I had enough time. Some new features such as the element system were added during development after the time and impact of the feature were assessed and decided were worth it. I understand now to keep to the core features the project needs first then re-evaluate if time allows or when those features are complete.

The major limitation of this project is that the product itself hasn't been confirmed to improve strategic skills in the target demographic. This is due to time constraints of the project and limitations of the users that were available for testing. The user experience testing was undertaken on university students so the enjoyment for 10–14-year-olds is not confirmed either. However, from the evidence gathered in the literature review I believe this product will improve strategic skills. The enjoyment of the game by the users in the user experience testing is a strong indicator that it will also be for the target demographic.

8.2 Future work

To develop this project further some new design features will have to be implemented. These mainly focus on assets to be made to increase clarity, for example by having more enemy types with specific elemental types of the player would be able to learn them and use the information in planning a strategy. Currently, with the limited assets available, this is limited to a few enemies but would be easy to expand with enemy variants. To further the skill development of the game some additional puzzles could be added. This would help players develop some skills used less in the combat section but still required for it. This would have to be researched and designed further which this project doesn't allow time for. Additionally, a longer form of testing is required to ensure that the aim of this project is met. Currently, relevant research suggests that this product will develop the correct skills however there is no concrete evidence. A longer testing phase is required for this, however the time limit and resources available for this project does not allow for this to occur.

9 References:

1. Gouveia, D., Lopes, D., and Vaz, C. (2011) *Serious gaming for experiential learning*. In: Proceedings of the Frontiers in Education Conference (FIE), IEEE. Available at: <https://doi.org/10.1109/fie.2011.6142778> (Accessed: 3 February 2026).
2. Saenz, M. and Cano, J. (n.d.) *Experiential learning through simulation games: An empirical study*. *International Journal of Electrical Engineering Education*. Available at: https://www.ijee.ie/arti-cles/Vol25-2/s12_ijee2144.pdf (Accessed: 3 February 2026)
3. Ho, S.-J., Hsu, Y.-S., Lai, C.-H., Chen, F.-H., and Yang, M.-H. (2022) *Applying game-based experiential learning to comprehensive sustainable development-based education*. *Sustainability*, 14(3), p.1172. Available at: <https://doi.org/10.3390/su14031172> (Accessed: 3 February 2026).
4. Bottino, R.M., Ferlino, L., Ott, M. and Tavella, M. (2007) *Developing strategic and reasoning abilities with computer games at primary school level*. *Computers & Education*, 49(4), pp.1272–1286. Available at: <https://doi.org/10.1016/j.compedu.2006.02.003> (Accessed: 16 February 2026).
5. Glass, B.D., Maddox, W.T. and Love, B.C. (2013) *Real-time strategy game training: emergence of a cognitive flexibility trait*. *PLoS ONE*, 8(8), p.e70350. Available at: <https://doi.org/10.1371/journal.pone.0070350> (Accessed: 12 March 2026).
6. Schell Games (n.d.) *The art of game design*. Available at: <https://schellgames.com/art-of-game-design> (Accessed: 23 March 2026).
7. Maule, A.J. and Edland, A.C. (2002) *The effects of time pressure on human judgement and decision making*. In: *Decision Making*, pp.203–218. Available at: <https://doi.org/10.4324/9780203444399-23> (Accessed: 23 March 2026).
8. Shaheen, A., Fotaris, P. and Fallahkhair, S. (2021) *A systematic review of using reflective design features in game-based learning*. Available at: <https://doi.org/10.34190/gbl.21.099> (Accessed: 23 March 2026).
9. Pesare, E., Roselli, T., Corriero, N. and Rossano, V. (2016) *Game-based learning and gamification to promote engagement and motivation in medical learning contexts*. *Smart Learning Environments*, 3(1). Available at: <https://doi.org/10.1186/s40561-016-0028-0> (Accessed: 7 April 2026).
10. Nadeem, M., Oroszlanyova, M. and Farag, W. (2023) *Effect of digital game-based learning on student engagement and motivation*. *Computers*, 12(9), p.177. Available at: <https://doi.org/10.3390/computers12090177> (Accessed: 7 April 2026).